

Solving the Algebraic Riccati Equation with the QZ Algorithm

Amp

A linear-quadratic regulator needs the stabilizing solution P of an algebraic Riccati equation, and P is the graph of the stable invariant subspace of a matrix built from the plant and the cost. That matrix can be the wrong object to hand to an eigensolver: forming it demands R^{-1} , and the eigenvectors it is usually read from carry almost no information when two eigenvalues nearly coalesce. This note follows the alternative the libraries actually use: keep the data unfactored in an extended Hamiltonian pencil, let the QZ algorithm reduce the pencil to an ordered generalized Schur form, and read P off the deflating subspace of the stable part. The running example is a unit mass on a unit spring with two cost knobs, each of which has a closed-form solution, so every route can be measured against an exact answer: the pencil route holds machine precision where the matrix route loses eight digits, and all routes inherit the same conditioning limit when an eigenvalue approaches the imaginary axis.

October 05, 2026

Contents

1. The problem, and the object we have to compute	3
2. The eigenvector basis is the wrong object to want	5
3. When forming the Hamiltonian is itself the loss	8
4. The extended pencil and its deflating subspace	9
5. What the QZ algorithm computes	11
6. What this buys, and what it does not	13
7. Sources and further reading	15
Bibliography	16

1. The problem, and the object we have to compute

A linear-quadratic regulator chooses u to minimize

$$J = \int_0^\infty (x^T Q x + u^T R u) dt, \quad \dot{x} = Ax + Bu, \quad (1)$$

with $Q = Q^T \geq 0$ and $R = R^T > 0$. If (A, B) is stabilizable and (C, A) is detectable, where $Q = C^T C$, the optimal feedback is $u = -R^{-1} B^T P x$ and the symmetric positive semidefinite P solves the continuous-time algebraic Riccati equation

$$A^T P + P A - P G P + Q = 0, \quad G = B R^{-1} B^T, \quad (2)$$

with the closed loop $A - GP$ stable. The whole job is to compute P . Written as a quadratic matrix equation, as above, that job has no obvious starting point: P has $n(n+1)/2$ unknowns, the equation is nonlinear, and where a solution would come from is not visible.

The equation is better read as a statement about a subspace. Define the Hamiltonian matrix

$$H = \begin{pmatrix} A & -G \\ -Q & -A^T \end{pmatrix}. \quad (3)$$

Then P satisfies Equation 2 exactly when the graph subspace of P ,

$$\mathcal{S}_P = \text{span} \begin{pmatrix} I_n \\ P \end{pmatrix}, \quad \text{satisfies} \quad H \begin{pmatrix} I_n \\ P \end{pmatrix} = \begin{pmatrix} I_n \\ P \end{pmatrix} (A - GP). \quad (4)$$

That is one line of block multiplication. The first block row of Equation 4 reads $A - GP = A - GP$, which is where the closed loop comes from. The second reads $-Q - A^T P = P(A - GP) = PA - PGP$, and moving everything to one side gives $-Q - A^T P - PA + PGP = 0$, which is Equation 2. So **the Riccati solution is the graph of an invariant subspace of H , and the closed loop is the action of H on that subspace**. Since the closed loop must be stable, the subspace is the one spanned by the eigenvectors of H belonging to its eigenvalues with negative real part: its **stable invariant subspace**.

From a basis of that subspace, the solution follows without solving Equation 2 again. Partition a basis of the stable subspace into an upper block X_1 and a lower block X_2 , both in $\mathbb{R}^{n \times n}$. Choose the basis to be the graph of P , so that $X_2 = P X_1$; then Equation 4 is exactly the invariance statement for that basis, and the solution is the matrix division

$$P = X_2 X_1^{-1} \quad (5)$$

provided X_1 is invertible. This elimination is the classical route to the Riccati solution, and it is also the shape of every method that follows: **a basis of the stable subspace comes first, and a division turns it into P** . The symplectic-matrix note in this repository derives the same identity with the symplectic form in the foreground, for the same plant ([Control/symplectic-matrix/symplectic-matrix.pdf](#), page 6); what this note needs from it is only the invariance statement Equation 4 and the division Equation 5.

The example, and two knobs. Take a unit mass on a unit spring, actuated by a force:

$$A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (6)$$

The unforced plant is an oscillator with eigenvalues $\pm i$, so it sits exactly on the stability boundary. Two cost choices on this one plant isolate two different difficulties, and both have closed-form solutions, which is why this plant is worth more here than a random matrix.

Both cost choices below keep the same symmetric structure, $P = \begin{pmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{pmatrix}$; only its three entries change.

Cheap state penalty: $Q = \varepsilon I$, $R = 1$. Then

$$\begin{aligned} p_{12} &= \sqrt{1 + \varepsilon} - 1, & p_{22} &= \sqrt{2p_{12} + \varepsilon}, \\ p_{11} &= p_{22}(1 + p_{12}). \end{aligned} \quad (7)$$

As $\varepsilon \rightarrow 0$ the penalty on the state vanishes, the gains go to zero, and the closed-loop eigenvalues tend to $\pm i$ from the left: the controller barely acts and the closed loop approaches the stability boundary. Measured: $\|P\| = 1.4143 \times 10^{-4}$ at $\varepsilon = 10^{-8}$, and the closed-loop real part is $-0.70711\sqrt{\varepsilon}$.

Cheap control: $Q = I$, $R = \rho$. With the same P ,

$$\begin{aligned} p_{12} &= \sqrt{\rho^2 + \rho} - \rho, & p_{22} &= \sqrt{\rho(2p_{12} + 1)}, \\ p_{11} &= p_{22}(1 + p_{12}/\rho). \end{aligned} \quad (8)$$

Now the **control** is nearly free, the gains saturate, and one closed-loop eigenvalue runs off to $-\infty$ while the other stays at -1 . Measured: $\|P\| = 1.0100$ at $\rho = 10^{-4}$ with closed-loop eigenvalues $\{-99.985, -1.0002\}$; at $\rho = 3 \times 10^{-10}$ the pair is $\{-57735, -1\}$ and $\|P\| = 1.0000$.

Each closed form is a solution of Equation 2 for its own data. The reproducing script verifies that by its residual – below 3.7×10^{-16} in every row it prints – and by the negative real part of the closed-loop spectrum. From here on there is an exact answer to measure every algorithm against, which is the only way to separate a conditioning effect from an algorithmic one.

Taken as given. Invertibility of R for the **statement** Equation 2, and stabilizability and detectability for existence and uniqueness of a stabilizing P [1]. The Riccati equation itself is not re-derived beyond Equation 4 the question this note answers is how the subspace in Equation 4 is computed when the matrix route is unattractive.

2. The eigenvector basis is the wrong object to want

The classical route to Equation 5 asks the eigensolver for the n eigenvectors of H belonging to eigenvalues with negative real part. That request is far more fragile than the subspace it is meant to describe.

The smallest example with the same mechanism. Perturb a Jordan block:

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ \varepsilon & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ \varepsilon & 1 \end{pmatrix}. \quad (9)$$

Its eigenvalues are $1 \pm \sqrt{\varepsilon}$, with unit eigenvectors proportional to $(1, \pm\sqrt{\varepsilon})$. A perturbation of size ε therefore splits a double eigenvalue into two eigenvalues at distance $2\sqrt{\varepsilon}$, and the two eigenvectors differ by an angle of order $\sqrt{\varepsilon}$: their overlap is

$$|v_1^T v_2| = \frac{1 - \varepsilon}{1 + \varepsilon}, \quad \text{so} \quad 1 - |v_1^T v_2| = \frac{2\varepsilon}{1 + \varepsilon}. \quad (10)$$

The script checks both statements against the computed eigenpairs at $\varepsilon = 10^{-4}, 10^{-8}, 10^{-12}$. The exponents are the point: the **eigenvalues** separate like $\sqrt{\varepsilon}$, while the **eigenvectors** coincide like $1 - O(\varepsilon)$. An eigenvector carries a direction, so at small ε two eigenvectors that an algorithm must tell apart differ by $O(\varepsilon)$ in direction – much less than the $\sqrt{\varepsilon}$ by which their eigenvalues separate. What is special to this 2×2 model is that it is not Hamiltonian and has a single pair; what survives is the $\sqrt{\varepsilon}$ splitting of a coalescing pair and the faster crowding of its eigenvectors.

The same mechanism in the running example. In the cheap-state-penalty knob the pairs near $\pm i$ coalesce as $\varepsilon \rightarrow 0$. The split of the pair that straddles the imaginary axis is exactly the gap $\sqrt{2p_{12} + \varepsilon} = \sqrt{2\varepsilon}(1 + O(\varepsilon))$, and its two members lie on opposite sides of the axis – one stable, one unstable. For the eigenvalue nearest $+i$ and its nearest twin, the script measures:

ε	$ \operatorname{Re} \lambda $	$1 - v_1^T v_2 $	$\operatorname{cond}(V)$	$\operatorname{relerr}_{\text{eig}}$	$\operatorname{relerr}_{\text{schur}}$	$\operatorname{relerr}_{\text{qz}}$
10^{-4}	7.071×10^{-3}	4.249×10^{-4}	7.071×10^1	5.33×10^{-14}	1.18×10^{-12}	2.11×10^{-13}
10^{-6}	7.071×10^{-4}	4.250×10^{-6}	7.071×10^2	3.34×10^{-11}	3.02×10^{-11}	8.81×10^{-11}
10^{-8}	7.071×10^{-5}	4.250×10^{-8}	7.071×10^3	8.31×10^{-9}	1.73×10^{-9}	3.71×10^{-9}
10^{-12}	7.071×10^{-7}	4.250×10^{-12}	7.071×10^5	2.83×10^{-5}	5.28×10^{-5}	8.34×10^{-5}

Table 1: The eigenvector route on the cheap-state-penalty knob. All errors are relative to the closed form Equation 7, computed by `scripts/verify_qz_riccati.m`. Inspect the last three columns: at every ε the three routes land within a factor of three of one another. Here relerr means $\|P_{\text{route}} - P\|/\|P\|$, and V is the full matrix of eigenvectors of H .

The numbers behind the table: the straddling pair's eigenvectors have overlap $1 - |v_1^T v_2| = 4.25\varepsilon$, so at $\varepsilon = 10^{-8}$ they agree to eight digits; the pair's real part is $|\operatorname{Re} \lambda| = 0.70711\sqrt{\varepsilon}$, so at $\varepsilon = 10^{-8}$ its stable and unstable members differ in real part by 1.4×10^{-4} ; and $\operatorname{cond}(V) = 0.707/\sqrt{\varepsilon}$, rising from 7.07×10^1 at $\varepsilon = 10^{-4}$ to 7.07×10^5 at $\varepsilon = 10^{-12}$.

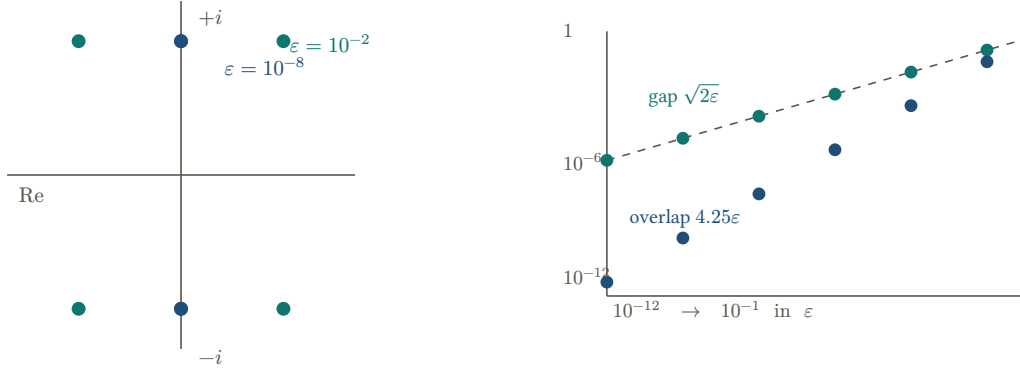


Figure 1: The spectrum of H in the cheap-state-penalty knob. Left: the four eigenvalues for $\varepsilon = 10^{-2}$ (teal) and $\varepsilon = 10^{-8}$ (navy). Each pair straddles the imaginary axis, and at this scale the $\varepsilon = 10^{-8}$ pair has collapsed onto it. Right: the two quantities that decide whether a pair can be told apart, on logarithmic axes, at $\varepsilon = 10^{-2}, 10^{-4}, \dots, 10^{-12}$. Inspect the slopes: the eigenvalue gap closes like $\sqrt{\varepsilon}$ (teal) while the eigenvectors crowd together like ε (navy).

Reading the table honestly. The obvious guess is that the eigenvector route is the inaccurate one. It is not, in this example. At $\varepsilon = 10^{-8}$ the three routes give relative errors 8.31×10^{-9} , 1.73×10^{-9} and 3.71×10^{-9} : all within a factor of three of 2.2×10^{-16} divided by ε , which is 2.2×10^{-8} . The reason is that the errors of two nearly parallel eigenvectors largely cancel in the ratio $X_2 X_1^{-1}$: here the stable subspace is a well-conditioned graph (the unit-norm graph basis of the stable subspace, and also the pair of stable eigenvectors, have smallest singular value 1.000 at $\varepsilon = 10^{-8}$). The eigenvector route's accuracy in this example is therefore not evidence that it is safe; it is a cancellation that the formulation does not promise. What the formulation **does** promise is weaker and more useful: the stable invariant subspace exists whenever no eigenvalue lies on the imaginary axis, with no requirement that H be diagonalizable or that a chosen basis be well separated.

Try it: Before reading on: from Table 1, does the eigenvector route return a less accurate P than the ordered-Schur route here?

Answer, immediately: no. The three columns are the same size to within a factor of three. The eigenvector route is fragile, not visibly wrong, in this example.

The repair, and it is not QZ. Laub's observation was that the **subspace** is what the problem defines, so one should get a basis of it from orthogonal transformations rather than from eigenvectors: reorder the real Schur form of H so the eigenvalues with negative real part lead, then read the graph off the leading columns [1]. He was blunt about the motivation: "the use of eigenvectors is often highly unsatisfactory from a numerical point of view and the present method uses the so-called and much more numerically attractive Schur vectors." Concretely, if

$$HU = UT, \quad U = \begin{pmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{pmatrix}, \quad T = \begin{pmatrix} T_{11} & T_{12} \\ 0 & T_{22} \end{pmatrix} \quad (11)$$

with U orthogonal and the eigenvalues of T_{11} in the open left half plane, then the leading n columns of U span the stable invariant subspace and, when U_{11} is invertible,

$$P = U_{21}U_{11}^{-1}. \quad (12)$$

That is the same division as Equation 5, with a basis that is orthonormal by construction and needs no eigenvector decomposition. Measured at $R = 1$ for $\varepsilon = 10^{-2}$ and 10^{-6} , this route and the pencil route of the next two sections agree to $\|P_q - P_s\|/\|P\| = 1.08 \times 10^{-14}$ and 7.30×10^{-11} : the machinery below does not change the answer here. It changes what the algorithm can be **given**.

3. When forming the Hamiltonian is itself the loss

The cheap-control knob breaks a different step: forming H . Its off-diagonal blocks are Q and G , and $G = BR^{-1}B^T$ calls for the inverse of the control weight. When R is small, the entries of G are large and their rounding error is amplified by the same small number.

Concretely, in the running example the $(2, 2)$ entry of G is $1/\rho$. Storing it as a floating-point number injects an absolute error of about ε_m divided by ρ , where $\varepsilon_m = 2.2 \times 10^{-16}$ is the unit roundoff, while the rest of H has entries of order one. So the matrix handed to the Schur route is already perturbed by that amount before the algorithm starts, and the relative error of the computed P is observed to track ε_m/ρ : the digits lost are, roughly, the digits of $1/\rho$.

ρ	$\ P\ $	relerr _{matrix}	relerr _{pencil}	DIGITS RECOVERED
10^{-4}	1.0100	3.09×10^{-13}	2.65×10^{-16}	3.1
10^{-6}	1.0010	1.09×10^{-10}	1.14×10^{-15}	5.0
3×10^{-8}	1.0002	1.56×10^{-9}	5.52×10^{-16}	6.5
3×10^{-10}	1.0000	2.21×10^{-7}	1.17×10^{-15}	8.3
0	—	no finite value	still a regular pencil	—

Table 2: The cheap-control knob: $Q = I$, $R = \rho$, measured against the closed form Equation 8. “matrix route” forms $G = BR^{-1}B^T$ and runs the ordered Schur route on H ; “pencil route” keeps B and R inside the extended pencil of Equation 14 and orders its generalized Schur form. Inspect the two error columns: they separate as ρ shrinks, and at $\rho = 0$ the matrix route has no value to compute.

The pencil route holds relative errors of 10^{-16} to 10^{-15} across the whole table – machine precision, with no degradation as ρ shrinks – while the matrix route tracks 2.2×10^{-16} divided by ρ to within a factor of ten. At $\rho = 0$ the comparison stops being about accuracy: $BR^{-1}B^T$ is not finite, so there is no matrix H to factorize, while the same data still defines a regular pencil whose QZ form exists. Measured for exactly that data: a 5×5 pencil with 2 finite and 3 infinite eigenvalues.

This is the documented reason the pencil formulation is standard. Van Dooren introduced the generalized eigenvalue route for precisely this data [2], and Benner and Sima state the point in one sentence: “the deflating subspace approach using the extended pencils yields better numerical accuracy if R is ill-conditioned as rounding errors introduced by forming R^{-1} are avoided” [3]. They make the same point on the discrete side: “the symplectic matrix can only be formed if A is nonsingular ... it is preferable to work with the symplectic pencil”, an objection that also applies to the descriptor matrix E .

What has changed conceptually. Nothing about the eigenvalues: the pencil of the next section has the same finite eigenvalues as H . What changes is that R is never inverted, and that a singular R – a singular **pencil** – is a representable object rather than a failure. That is what the QZ algorithm, unlike a QR iteration on a single matrix, is built to process.

4. The extended pencil and its deflating subspace

The eigenvalue problem that keeps the data unfactored is a **generalized** one: find λ and $v \neq 0$ with

$$Mv = \lambda Nv, \quad \text{that is} \quad (M - \lambda N)v = 0. \quad (13)$$

The pencil $M - \lambda N$ carries a **pair** (α, β) instead of a single number: $\lambda = \alpha/\beta$ when $\beta \neq 0$, and the pair $(\alpha, 0)$ is an infinite eigenvalue, a legitimate answer rather than an error. The pencil is called regular when $\det(M - \lambda N)$ does not vanish identically. Nothing here requires N , or any block of it, to be invertible – exactly the property that forming $G = BR^{-1}B^T$ destroyed.

For the Riccati equation the pencil is built by keeping B and R as blocks. With the state split as before and the control carried along as a third block, take

$$M - \lambda N = \begin{pmatrix} A & 0 & B \\ -Q & -A^T & 0 \\ 0 & B^T & R \end{pmatrix} - \lambda \begin{pmatrix} I & 0 & 0 \\ 0 & I & 0 \\ 0 & 0 & 0 \end{pmatrix}. \quad (14)$$

A	$-G = -BR^{-1}B^T$
$-Q$	$-A^T$

A	0	B
$-Q$	$-A^T$	0
0	B^T	R

H : the shaded blocks are **formed**; R^{-1} touches every entry of G

$M - \lambda N$: teal blocks are the data as given; no inverse

Figure 2: The two objects for the same data. Left: the matrix route forms G first, which means applying R^{-1} and merging the result with B . Right: the extended pencil keeps A , B , Q , R as separate blocks and adds only identity blocks, so no inverse is formed anywhere. Inspect which route can still be assembled when R is singular: only the right one.

Why this pencil answers the Riccati equation is a one-line elimination. Write a null vector with blocks x_1, x_2, u in the order of Equation 14. The third block row is $B^T x_2 + Ru = 0$, so $u = -R^{-1}B^T x_2$. The second is $-Qx_1 - A^T x_2 = \lambda x_2$, and the first becomes $Ax_1 - Gx_2 = \lambda x_1$. Setting $x_2 = Px_1$ in the first gives $\lambda = A - GP$ applied to x_1 ; putting the same relation in the second gives $0 = Q + A^T P + P(A - GP) = Q + A^T P + PA - PGP$, which is Equation 2. So the finite eigenvalues of Equation 14 are the eigenvalues of H , and the graph subspace of the Riccati solution, Equation 4, is a **deflating subspace** of the pencil: a subspace Z with $MZ = NZS$ for some S , the generalization of an invariant subspace to a pencil. Note what the elimination above is **not**: it is not what the algorithm does. Solving $u = -R^{-1}B^T x_2$ would invert R again, so the algorithm never solves for u ; it removes the third block by an orthogonal transformation instead. The script's check of this whole construction is indirect but decisive: at $R = 1$, where the pencil reduces to the standard problem, the pencil route and the ordered-Schur route on H agree to 1.08×10^{-14} relative. SLICOT documents the same reduction: "a standard eigenproblem is solved in the continuous-time case if G is given" [4].

Deflation. The pencil Equation 14 is $(2n + m) \times (2n + m)$ where m is the number of controls. When R is nonsingular it has exactly m infinite eigenvalues, all of them contributed by the last block, so only $2n$ of them are finite; before ordering anything, the algorithm compresses that cost direction away. Let M_c be the last m columns of M , that is the stacked blocks B , 0 , R , and take a QR decomposition

$$M_c = Q \begin{pmatrix} R_1 \\ 0 \end{pmatrix}, \quad \text{then} \quad \tilde{M} - \lambda \tilde{N} = Q_2^T (M - \lambda N) \begin{pmatrix} I_{2n} \\ 0 \end{pmatrix}, \quad (15)$$

where Q_2 is Q with its first m columns dropped. The result is a $2n \times 2n$ pencil with the same finite eigenvalues and none infinite. This is van Dooren's reduction; `scipy's solve_continuous_are` performs it with the comment "deflate the pencil to $2m \times 2m$ ala Ref.1, eq.(55)" [5]. The script measures the counts for the running example, where $m = 1$: 4 finite and 1 infinite at $R = 10^{-6}$, becoming 4 and 0 after the deflation. At $R = 0$ the same data gives 2 finite and 3 infinite, of which the deflated 4×4 pencil keeps 2 – harmless, because a pair with $\beta = 0$ never satisfies $\text{Re}(\alpha/\beta) < 0$ and so is never selected.

The deflated second matrix is not the identity, and it inherits the conditioning of R : its smallest singular value is 7.07×10^{-1} at $R = 1$, 1.00×10^{-6} at $R = 10^{-6}$, and 9.46×10^{-9} for $R = \text{diag}(1, 10^{-8})$ at a second size the script checks. QZ touches that matrix only with unitary transformations, so it never inverts it. That is section 3's eight-digit loss seen at the level of the algorithm: forming $G = BR^{-1}B^T$ is inverting this block.

5. What the QZ algorithm computes

The QZ algorithm is the generalized eigenvalue problem’s answer to the QR algorithm [6]: it reduces a **pair** of matrices to triangular form with two unitary transformations,

$$Q^T(M - \lambda N)Z = S - \lambda T, \quad S, T \text{ upper triangular, } Q, Z \text{ unitary,} \quad (16)$$

the pairs (S_{ii}, T_{ii}) on the diagonals being the eigenvalues α/β . Because Q and Z are unitary, no matrix is inverted and no column of the pencil is privileged. Moler and Stewart’s abstract states the property the Riccati problem needs: “no inversions of B or its submatrices are used” – their B is the pencil’s second matrix, which is N here. In floating point the pencil is balanced first, which is what makes the computed subspace accurate for badly scaled data [7].

The second ingredient is **ordering**. A QZ driver such as LAPACK’s `dggss`, reachable as `ordqz` in Octave and MATLAB and as `scipy.linalg.ordqz`, takes a selection rule and moves the selected pairs to the leading diagonal blocks by further unitary transformations. The rule used for the Riccati equation selects the pairs with $\text{Re}(\alpha/\beta) < 0$ (Octave’s `'lhp'`, LAPACK’s `sort` keyword). After ordering, the first n columns of Z are a unitary basis of the stable deflating subspace, and the solution is read off them.

S_{11}, T_{11}	S_{12}, T_{12}	U_{11}	U_{12}
0	S_{22}, T_{22}	U_{21}	U_{22}

n stable pairs first, by unitary reordering

$P = U_{21}U_{11}^{-1}$, a solve rather than an inverse

Figure 3: The ordered generalized Schur form. Left: S and T after QZ, with the selected stable pairs moved to the leading block by unitary transformations. Right: the conforming partition of Z . Only the final, small system $U_{11}^T P^T = U_{21}^T$ touches U_{11} , and it is solved by substitution; the defect $\|U_{11}^T U_{21} - U_{21}^T U_{11}\|$ measures the symmetry that P should have.

The P that comes out is symmetric only up to the computation. `scipy`’s implementation checks exactly this and refuses to return a plausible-looking answer when the check fails: its docstring notes that “the fail conditions are linked to the symmetry of the product $U_2 U_1^{-1}$ and condition number of U_1 ”, and the source raises `LinAlgError` with the message “the associated Hamiltonian pencil has eigenvalues too close to the imaginary axis” when the antisymmetry defect of $U_{11}^T U_{21}$ exceeds a threshold [5].

How much does the defect tell you? Less than an error bound. Perturbing the extended pencil by 100 random roundings and re-running the route, the script measures both the true relative error of P and the defect:

ε	$\text{relerr}(P)$	DEFECT
10^{-4}	1.91×10^{-12}	3.45×10^{-14}
10^{-6}	1.76×10^{-10}	3.21×10^{-13}
10^{-8}	1.66×10^{-8}	3.01×10^{-12}
10^{-10}	1.31×10^{-6}	2.98×10^{-11}

The defect moves with the error – both grow as ε falls, the error by nearly six orders of magnitude and the defect by three – but the defect is 10^1 to 10^5 times **smaller** than the error, so it is not a bound and not an estimate. Its value is as a tripwire: it costs almost nothing, it is not fooled by a badly conditioned

subspace, and `scipy` turns it into a refusal instead of a wrong answer.

Finally, this is what a library routine actually is. MATLAB's `care`, SLICOT's `SB020D` and `scipy`'s `solve_continuous_are` all call the same generalized Schur machinery. The engineering content is choosing the pencil and reading the deflating subspace; the QZ iteration itself is the one piece of this note one should **not** write by hand.

6. What this buys, and what it does not

What it buys, stated so it can be checked. For the running example with $Q = I$ and $R = \rho$, forming $G = BR^{-1}B^T$ in double precision and running the ordered Schur route returns P with relative error 3.09×10^{-13} at $\rho = 10^{-4}$, 1.09×10^{-10} at $\rho = 10^{-6}$, 1.56×10^{-9} at $\rho = 3 \times 10^{-8}$ and 2.21×10^{-7} at $\rho = 3 \times 10^{-10}$; the deflated QZ route on the extended pencil returns 2.65×10^{-16} , 1.14×10^{-15} , 5.52×10^{-16} and 1.17×10^{-15} on the same data. At $\rho = 0$ the matrix route has nothing to compute, while the pencil is regular with 2 finite and 3 infinite eigenvalues. Every number in this paragraph is printed by `scripts/verify_qz_riccati.m`, so it can be falsified by running it.

What it is not. It is not an answer to near-axis eigenvalues. The tempting claim is that QZ is more accurate than the eigenvector or ordered-Schur routes when the Hamiltonian spectrum straddles the imaginary axis. In the cheap-state-penalty knob it is not: the relative errors of the three routes at $\varepsilon = 10^{-8}$ are 8.31×10^{-9} , 1.73×10^{-9} and 3.71×10^{-9} , all within a factor of three of 2.2×10^{-16} divided by ε , and at $\varepsilon = 10^{-12}$ they are 2.83×10^{-5} , 5.28×10^{-5} and 8.34×10^{-5} . The limit is the data, not the algorithm: the pair straddling the axis has a gap that closes like $\sqrt{\varepsilon}$, the absolute error in P is of order $\varepsilon_m / \text{gap}$, and since $\|P\|$ itself is only of order $\sqrt{\varepsilon}$, the relative error grows like $1/\varepsilon$. At $\varepsilon = 10^{-16}$ the problem's own relative condition number is of order 10^{16} and no route returns a meaningful relative answer.

Two further statements that are **not** true, worth ruling out because both are natural guesses after seeing the pencil:

- QZ does not make an eigenvalue on the imaginary axis tractable. If an eigenvalue of H lies exactly on the axis, the stable and unstable deflating subspaces are not separated, there is no stabilizing P , and the best closed loop available is only marginally stable; scipy raises rather than returning a value. The tool for spectra close to or on the axis is a structure-preserving Hamiltonian or symplectic method, which computes a **structured** decomposition and keeps its accuracy [8] – genuinely outside this note's scope.
- The pencil is not a different answer. With R invertible its finite eigenvalues are those of H , and the computed P agrees with the matrix route to 1.08×10^{-14} relative at $\varepsilon = 10^{-2}$. What the pencil changes is the class of data the algorithm can accept.

The recipe, once. Assemble the extended pencil Equation 14 balance it; deflate the control block by QR as in Equation 15 run an ordered QZ with $\text{Re}(\alpha/\beta) < 0$; set $P = U_{21}U_{11}^{-1}$ by substitution; check the antisymmetry defect and $\text{cond}(U_{11})$; if a solution is wanted to maximal accuracy, refine it by a Newton step, which Arnold and Laub give for the generalized problem [9]. Everything except the last solve is unitary, and the one matrix that must be inverted is the one the answer itself depends on.

What this note did not attempt. The discrete-time equation (same deflating-subspace recipe with the symplectic pencil, and no A^{-1}), descriptor systems and the cross term S (both are extra blocks in the same pencil), large sparse problems (a different class of methods), and error estimation for P (SLICOT's `SB02RD` provides it; the defect of Figure 3 is not it).

KNOB	WHAT DEGRADES	WHAT LIMITS ACCURACY
cheap state penalty $Q = \varepsilon I$	the eigenvectors of the coalescing pair: overlap $1 - O(\varepsilon)$, gap $\sqrt{\varepsilon}$	the gap: relative error of order $1/\varepsilon$ times ε_m
cheap control $R = \rho$	$G = BR^{-1}B^T$: the rounding of R^{-1} is baked into H before the algorithm runs	nothing, here: the pencil route holds 10^{-16} to 10^{-15} down to $\rho = 3 \times 10^{-10}$

Table 3: The two knobs, the route that degrades in each, and the limit that survives both. The matrix route and the pencil route are the same algorithm up to which data reaches the QZ; the conditioning limit is a property of the Riccati equation, not of either route.

7. Sources and further reading

The classical eigenvector route to Equation 5, the fragility of that basis, and the ordered Schur repair of section 2 are Laub's [1]; the same paper proves the symmetry of P that the defect check of section 5 appeals to. The extended pencil, the deflating-subspace formulation and the QR deflation Equation 15 are van Dooren's [2], which is also the reference `scipy's solve_continuous_are` cites for that step. The generalized eigenproblem framework for Riccati equations – including the balancing and iterative refinement that make it a software package, partly out of scope here – is Arnold and Laub [9]. The QZ algorithm itself, and the “no inversions” property quoted in section 5, is Moler and Stewart [6]. The statement used in section 3, that the extended pencil avoids forming R^{-1} , together with the remark that the symplectic matrix form needs A invertible while the pencil does not, is Benner and Sima's survey of the SLICOT solvers [3]; the balancing step of section 5 is Benner [7]. The software behaviour in sections 4 and 5 – the extended pencil with its E and S blocks, the deflation, `ordqz` with the left-half-plane sort, and the `LinAlgError` raised when the pencil's eigenvalues approach the axis – is `scipy's` documentation and source [5], and the “standard eigenproblem if G is given” statement is SLICOT's `SB020D` documentation [4]. The structure-preserving alternative mentioned in section 6 is Benner, Mehrmann and Xu [8].

Not consulted, and offered only as pointers for going further: Golub and Van Loan's **Matrix Computations** for the generalized Schur decomposition as textbook material, and the CAREX/DAREX benchmark collections (Abels and Benner, SLICOT working notes) for larger test problems.

All quoted numbers come from `scripts/verify_qz_riccati.m`, which is also the check that the two closed forms of section 1 solve the Riccati equation for their own data.

Bibliography

- [1] A. J. Laub, “A Schur Method for Solving Algebraic Riccati Equations,” *IEEE Transactions on Automatic Control*, vol. 24, no. 6, pp. 913–921, 1979, doi: 10.1109/TAC.1979.1102178[◦].
- [2] P. Van Dooren, “A Generalized Eigenvalue Approach for Solving Riccati Equations,” *SIAM Journal on Scientific and Statistical Computing*, vol. 2, no. 2, pp. 121–135, 1981, doi: 10.1137/0902010[◦].
- [3] P. Benner and V. Sima, “Solving Algebraic Riccati Equations with SLICOT,” in *CD-ROM Proceedings of the 11th Mediterranean Conference on Control and Automation (MED 2003)*, Rhodes, Greece, 2003. [Online]. Available: <https://cscproxy.mpi-magdeburg.mpg.de/mpcsc/benner/pub/med03.pdf>[◦]
- [4] SLICOT Working Group, “SB02OD: Solution of Continuous- or Discrete-Time Algebraic Riccati Equations (Generalized Schur Vectors Method).” [Online]. Available: <https://www.slicot.org/objects/software/shared/doc/SB02OD.html>[◦]
- [5] SciPy developers, “scipy.linalg.solve_continuous_are, SciPy v1.18.0 reference.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve_continuous_are.html[◦]
- [6] C. B. Moler and G. W. Stewart, “An Algorithm for Generalized Matrix Eigenvalue Problems,” *SIAM Journal on Numerical Analysis*, vol. 10, no. 2, pp. 241–256, 1973, doi: 10.1137/0710024[◦].
- [7] P. Benner, “Symplectic Balancing of Hamiltonian Matrices,” *SIAM Journal on Scientific Computing*, vol. 22, no. 5, 2001, doi: 10.1137/S1064827500367993[◦].
- [8] P. Benner, V. Mehrmann, and H. Xu, “A Numerically Stable, Structure Preserving Method for Computing the Eigenvalues of Real Hamiltonian or Symplectic Pencils,” *Numerische Mathematik*, vol. 78, no. 3, pp. 329–358, 1998, doi: 10.1007/s002110050315[◦].
- [9] W. F. Arnold and A. J. Laub, “Generalized Eigenproblem Algorithms and Software for Algebraic Riccati Equations,” *Proceedings of the IEEE*, vol. 72, no. 12, pp. 1746–1754, 1984, doi: 10.1109/PROC.1984.13083[◦].